Experiment No. 10

AIM
Implement the embedding wallet (Metamask) and transaction using Solidity

THEORY

1. MetaMask

MetaMask is a **cryptocurrency wallet and browser extension** that allows users to interact with blockchain networks and decentralized applications (DApps). It manages blockchain accounts and allows users to approve or reject transactions.

In this experiment, MetaMask is used to connect the user's Ethereum account with Remix IDE and to authorize smart-contract transactions.

2. Testnet

A **testnet** is a blockchain network used for development and testing purposes. It works similarly to a real blockchain but uses **test cryptocurrency**, which has no real monetary value.

In this experiment, the **Ethereum Sepolia Testnet** was used. Sepolia ETH was obtained from a faucet and used to pay the gas fees for deploying and interacting with the smart contract.

3. Remix IDE

Remix IDE is a web-based development environment used to write, compile, deploy, and interact with Solidity smart contracts.

Remix was connected to MetaMask using the **Browser Extension / Sepolia Testnet - MetaMask** environment.

4. Smart Contract

A smart contract is a program stored on a blockchain that executes according to predefined rules. In this experiment, a simple StudentRecord contract was created to store a student's name and roll number.

The contract contains:

- name — stores the student's name.
- rollNo — stores the student's roll number.
- setStudent() — changes the stored student information.
- getStudent() — retrieves the stored student information.

5. Transactions and Gas

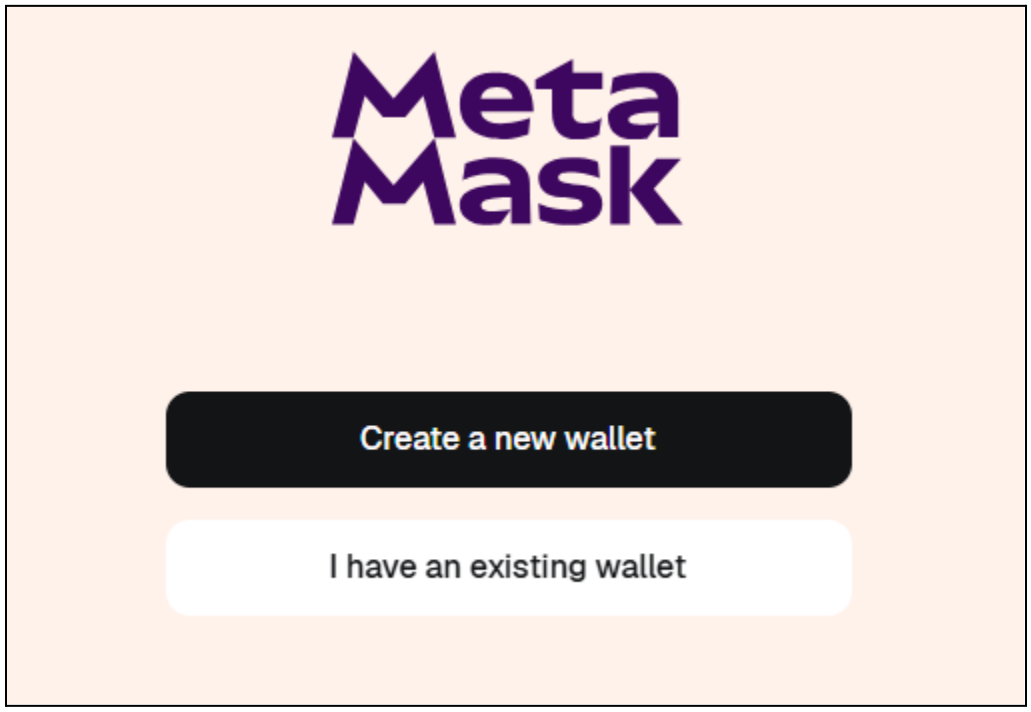
A transaction is required when a blockchain state is changed. Therefore, deploying the contract and calling setStudent() require a transaction and gas fees.

The getStudent() function only reads information from the blockchain, so it does not require a new transaction or gas payment.

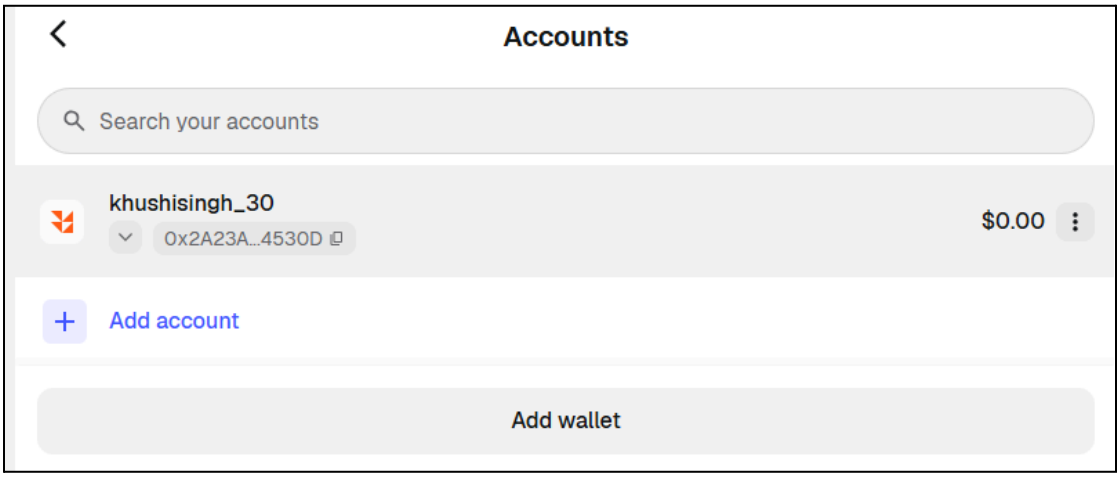
PROCEDURE / STEPS

Step 1: Set Up MetaMask

1. Install the **MetaMask browser extension**.
2. Create a new MetaMask wallet or import an existing wallet.



3. Create/select the account to be used for the experiment.



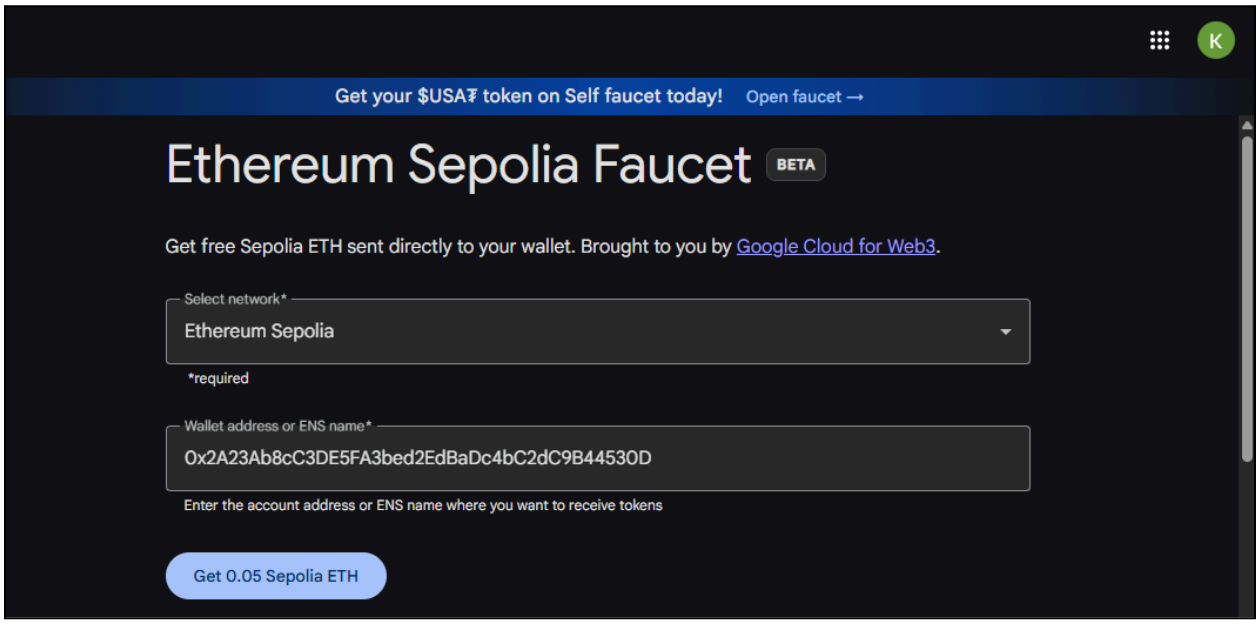
4. The MetaMask account used in the experiment was:

0x2A23Ab8cC3DE5FA3bed2EdBaDc4bC2dC9B44530D

5. Open MetaMask and select the **Ethereum Sepolia Testnet**.

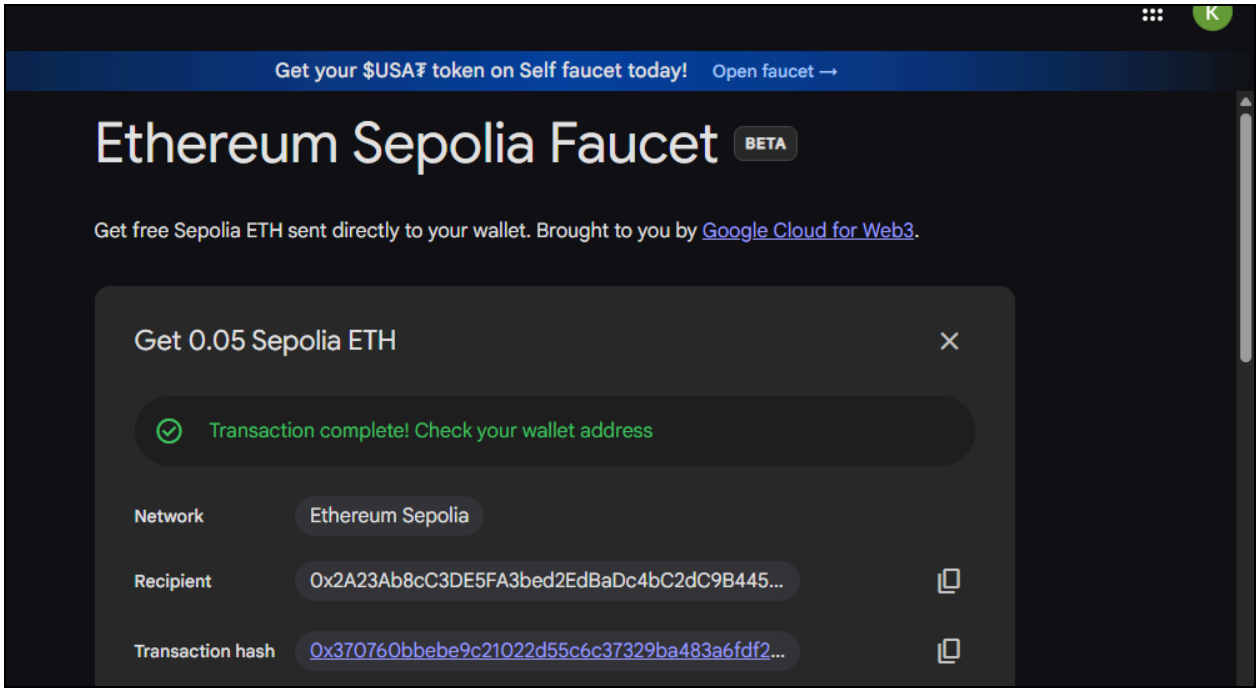
Step 2: Obtain Sepolia Test ETH

1. Open a Sepolia ETH faucet.
2. Enter the MetaMask wallet address.

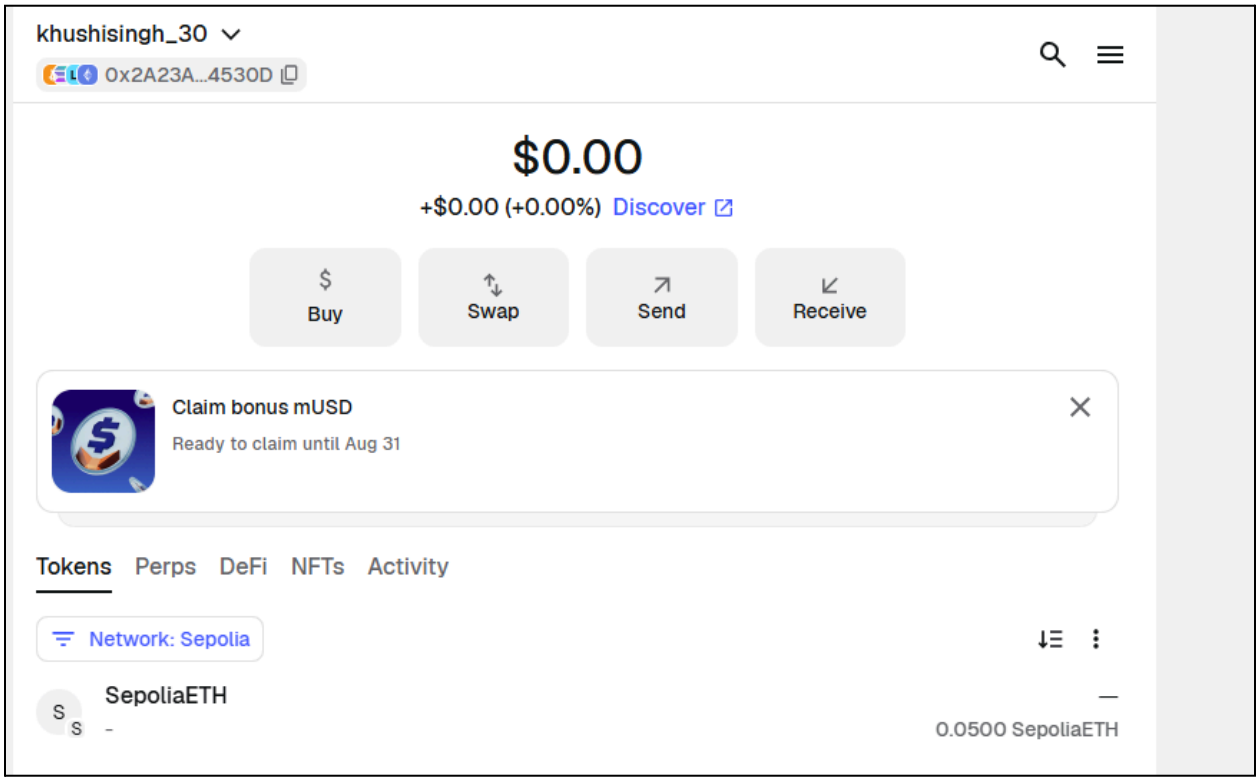


3. Request test ETH.
4. The wallet received approximately:

0.05 Sepolia ETH



5. This test ETH was used to pay gas fees during the experiment.



Step 3: Open Remix IDE

1. Open Remix IDE in a web browser.

2. Create a new Solidity file.
3. Name the file
4. Write the following Solidity program.

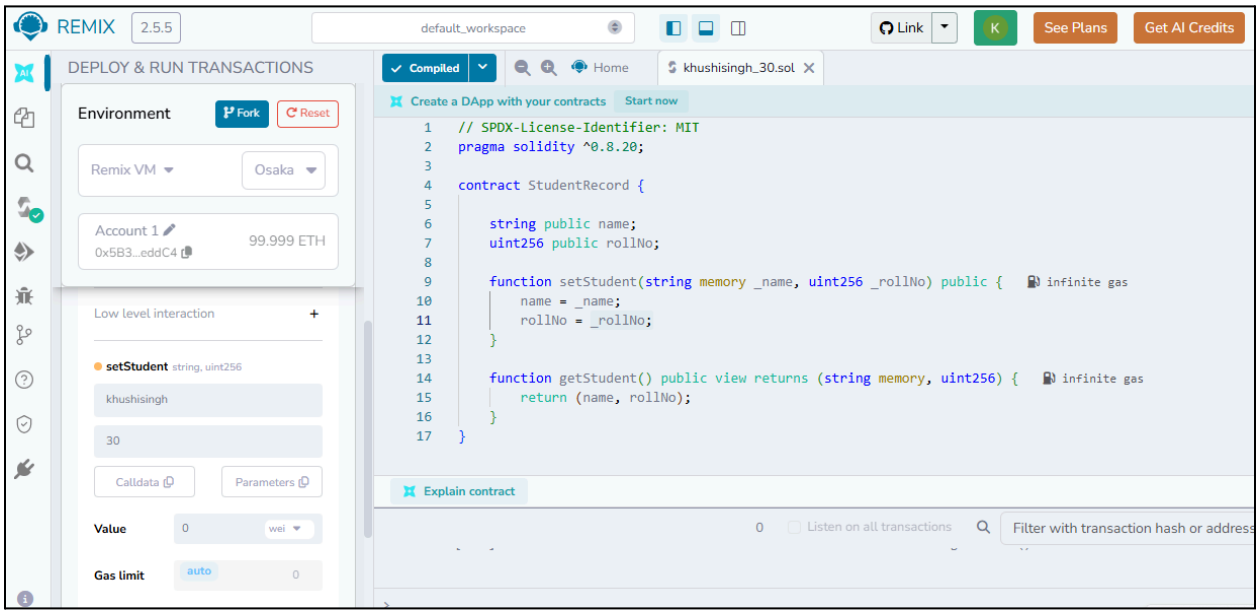
PROGRAM

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StudentRecord {
    string public name;
    uint256 public rollNo;

    function setStudent(string memory _name, uint256 _rollNo) public {
        name = _name;
        rollNo = _rollNo;
    }

    function getStudent() public view returns (string memory, uint256) {
        return (name, rollNo);
    }
}
```



Step 4: Compile the Smart Contract

Step 5: Connect MetaMask with Remix

1. Open **Deploy & Run Transactions** in Remix.
2. In the **Environment** dropdown, select:

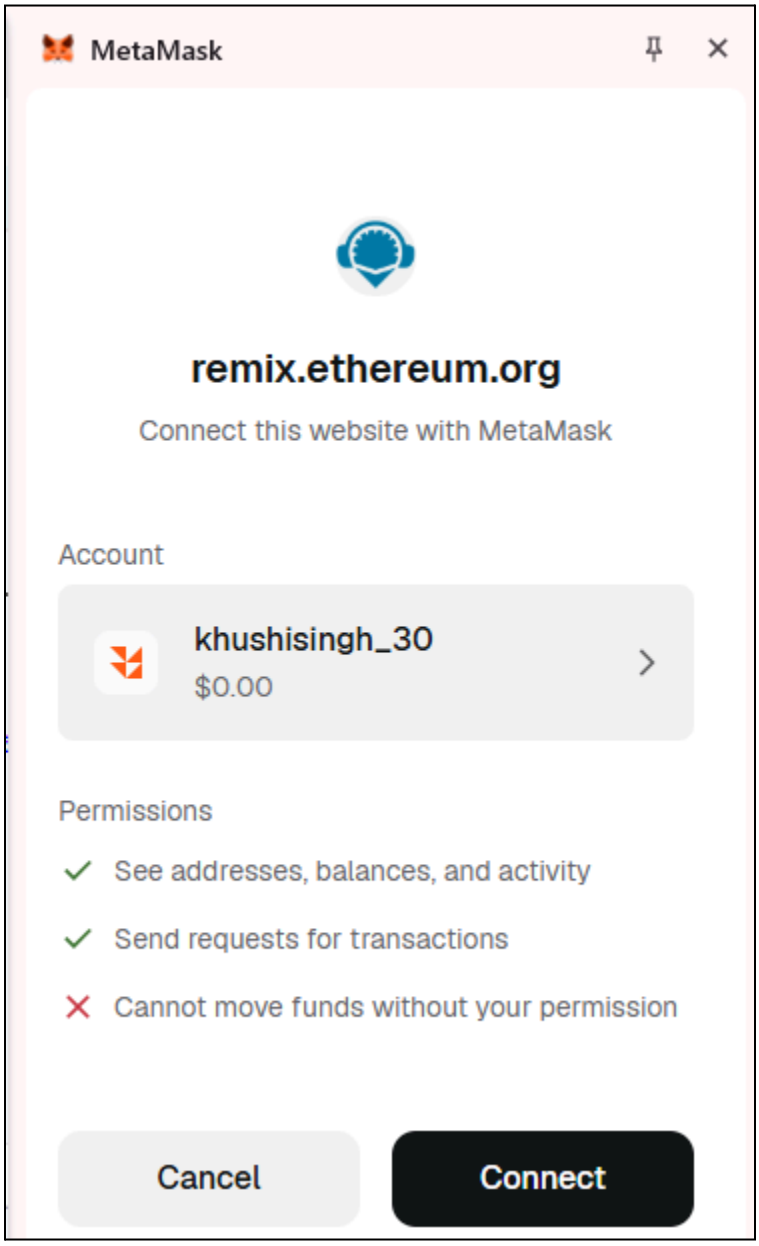
Sepolia Testnet - MetaMask

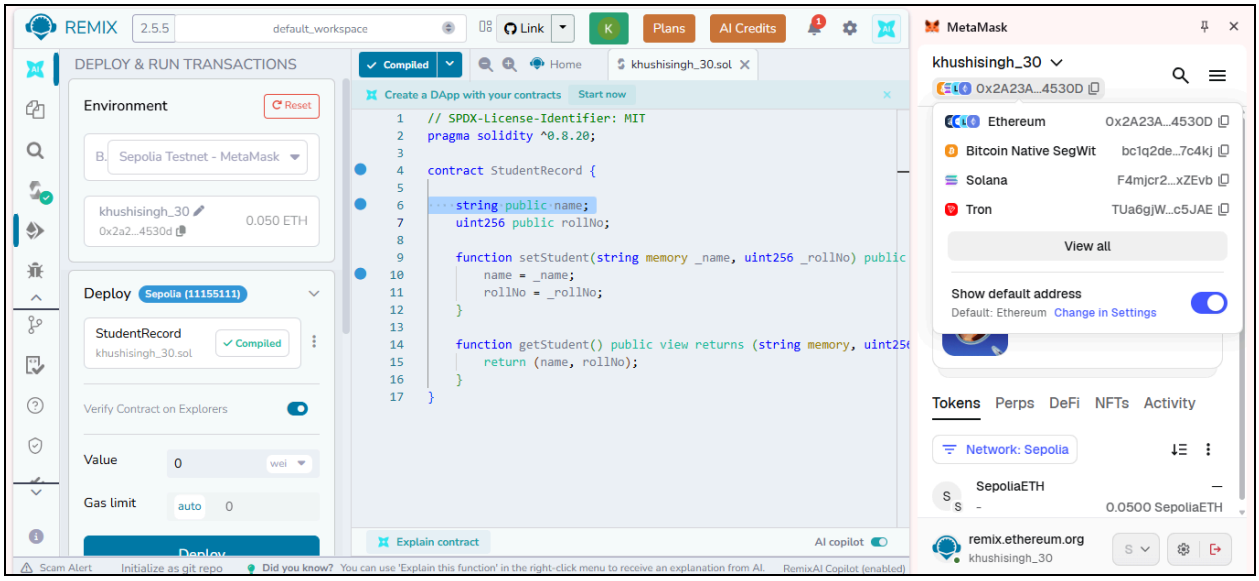
3. MetaMask opens and asks for permission to connect with Remix.
4. Select the MetaMask account.
5. Click **Next** and then **Connect**.
6. Verify that Remix shows the MetaMask account and Sepolia network.

The Remix environment should show:

Sepolia Testnet - MetaMask

and the connected MetaMask account.



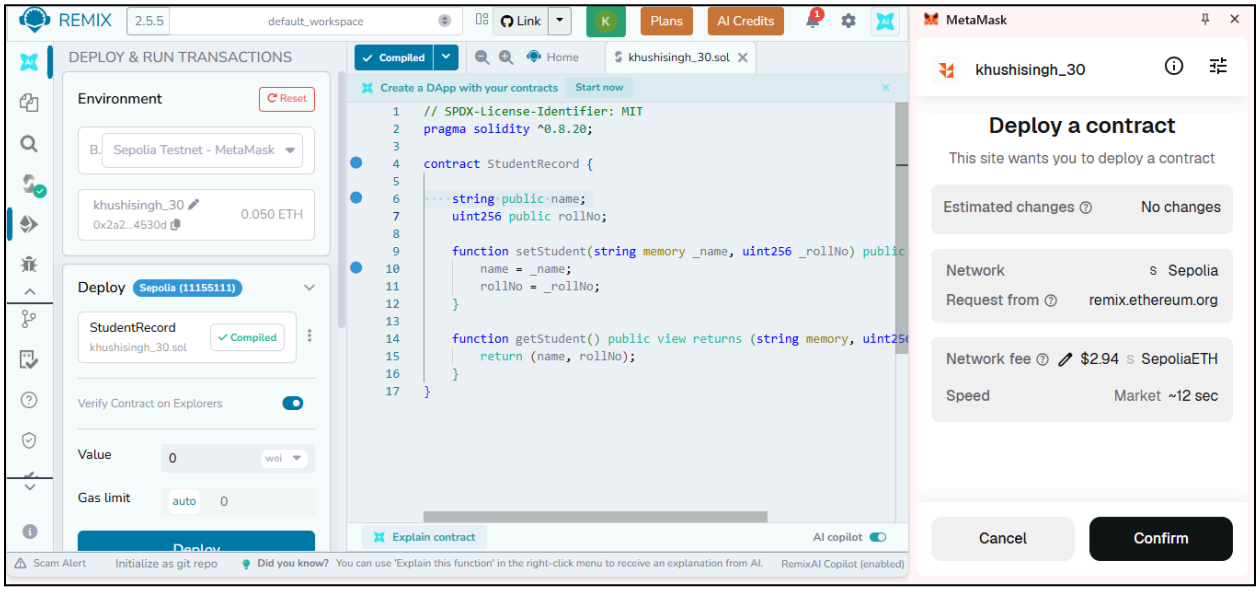


Step 6: Deploy the Smart Contract

1. In **Deploy & Run Transactions**, select StudentRecord.
2. Keep the **Value** as:

0 wei

3. Keep the gas limit as **auto**.
4. Click **Deploy**.
5. MetaMask opens with a transaction confirmation.
6. Verify that the network is **Sepolia Testnet**.
7. Click **Confirm** in MetaMask.



8. Wait for the transaction to be confirmed.

After successful deployment, the contract appears under:

Deployed Contracts

with its contract address.

Step 7: Verify Contract Deployment

The deployment transaction was checked on **Sepolia Etherscan**.

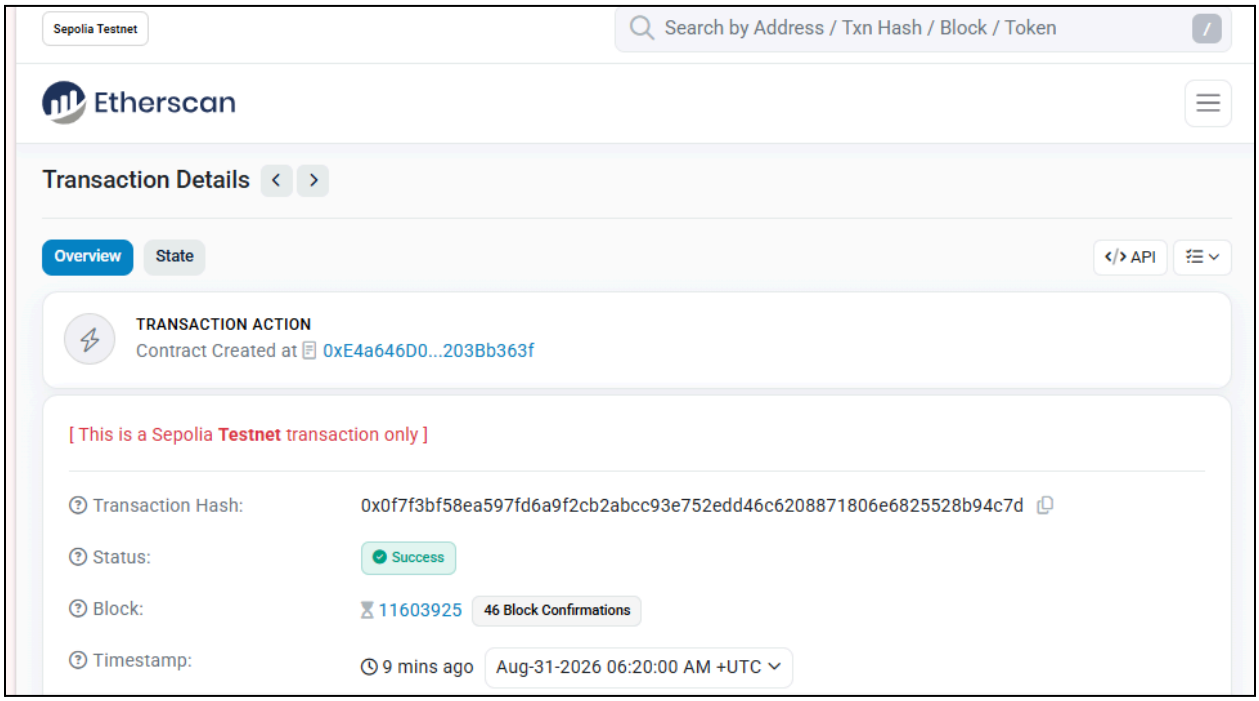
The transaction showed:

Network: Sepolia Testnet
Status: Success

The contract was successfully created on the blockchain.

The deployment transaction hash recorded during the experiment was:

0x0f7f3bf58ea597fd6a9f2cb2abcc93e752edd46c6208871806e6825528b94c7d



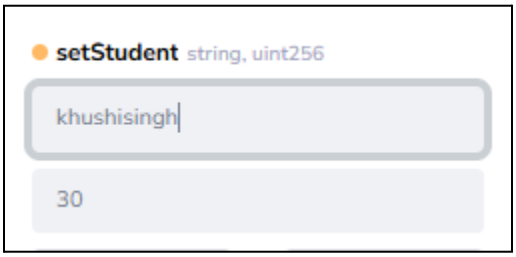
Step 8: Interact with the Deployed Contract

1. Expand the **StudentRecord** contract under **Deployed Contracts**.
2. The following functions are displayed:

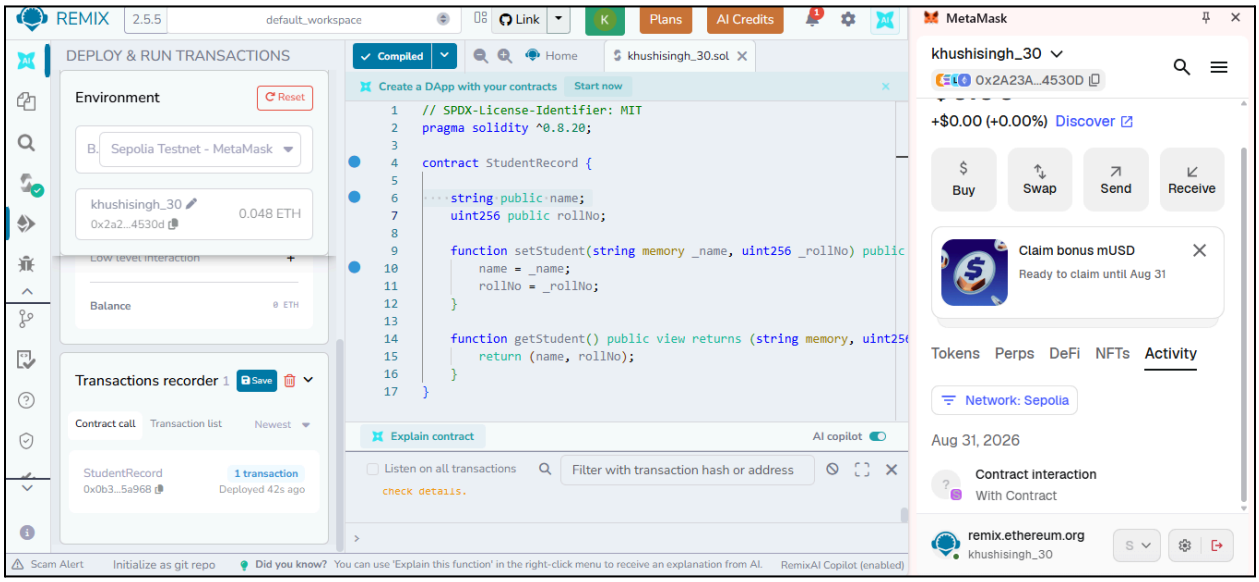
name
rollNo
setStudent
getStudent

3. Initially, name and rollNo contain their default values.

Step 9: Perform setStudent() Transaction

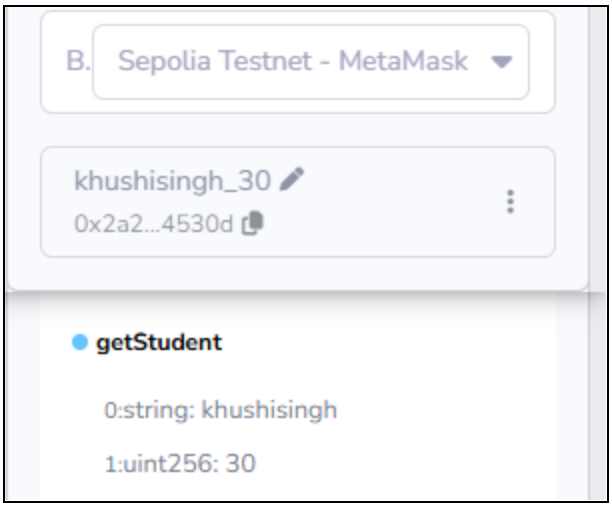


This transaction changes the state of the smart contract and therefore requires gas.



Step 10: Retrieve Student Information

Output:



This confirms that the information was successfully stored in the smart contract.

CONCLUSION

The MetaMask wallet was successfully connected to **Remix IDE** through the **Ethereum Sepolia Testnet**. A Solidity-based StudentRecord smart contract was compiled and deployed successfully. A state-changing transaction was performed using setStudent() through MetaMask, and the stored student details were retrieved using getStudent().

Thus, the experiment successfully demonstrated **MetaMask wallet integration with Remix IDE, smart contract deployment, blockchain transactions, gas usage, and verification of transactions on the Sepolia Testnet**.